

PROJECT REPORT

Cognton-AI-SIG-IDS

Signature-Based Network Traffic Detection Engine in Rust

1. Introduction

Cognton-AI-SIG-IDS is a signature-based network traffic detection engine written in Rust. It analyzes packet capture (pcap) files, extracts network flows, enriches them with hostnames and URLs, matches indicators of compromise (IOCs) against threat-intelligence feeds, and classifies each connection as Benign, Suspicious, or Malicious. The tool was developed for the Cognton AI Rust assignment and mirrors detection-engine patterns used on Cognton’s agentic platform.

Key goals: offline-capable analysis (no root required), O(1) signature matching, real threat-intel integration (AbuseIPDB + URLhaus), and clean CLI + JSON reporting.

2. Objectives & Technology Choice

2.1 Project Objectives

- 1. Ingest and parse pcap files into structured flow records.
- 2. Aggregate packets into per-connection flows and enrich with HTTP Host, DNS query names, and TLS SNI.
- 3. Load threat signatures from AbuseIPDB and URLhaus (with local cache) or offline fixtures.
- 4. Perform O(1) matching of IPs, domains, and URLs against a pre-indexed signature set.
- 5. Assign verdicts (Benign / Suspicious / Malicious) and produce CLI summary + JSON export.

2.2 Why Rust?

- Strong static typing – reduces runtime errors when handling packet fields and IOCs.
- Memory safety – eliminates buffer overflows and use-after-free when parsing untrusted pcap data.
- Concurrency & ownership model – safe parallel processing without data races.
- Low-level control – efficient access to memory, networking, files, and system resources.
- High performance & reliability – compiled language with zero-cost abstractions, no GC pauses, and compile-time guarantees.

3. System Architecture

The engine follows a clear linear pipeline: **Ingest** → **Aggregate** → **Load Signatures** → **Match** → **Classify** → **Report**. CLI (clap) accepts --pcap and --output flags. Offline mode is controlled by the SIG_IDS_OFFLINE environment variable.

3.1 Module Responsibilities

Module	Responsibility
types.rs	Shared structs & enums: Flow, Signature, Verdict, Detection, locKind
ingestion.rs	Parses pcap → per-packet Flow; extracts hostname via HTTP Host, DNS qname, TLS SNI
aggregate.rs	Merges per-packet records into per-connection flows (direction-independent)
signatures.rs	AbuseIPDB + URLhaus clients, local JSON cache (1-hour TTL), offline fixture support
detector.rs	Builds HashMap-indexed SignatureIndex; O(1) IP / domain / URL matching
classifier.rs	Verdict assignment (confidence ≥ 90 → Malicious, else Suspicious) + simulated IPS block log
report.rs	CLI summary counts and pretty-printed JSON export of all detections
main.rs	CLI argument parsing (clap) and full pipeline wiring

4. Core Implementation Highlights

4.1 Packet Parsing – `flow_from_ethernet_frame()`

Located in `ingestion.rs`. Uses `ethers::SlicedPacket` to decode Ethernet → IPv4/IPv6 → TCP/UDP. Extracts 5-tuple (src/dst IP, ports, protocol). Optionally enriches the Flow with:

- HTTP Host header and reconstructed URL (http/https + path)
- DNS query name (when port 53 is involved)
- TLS SNI from ClientHello handshake records (enables hostname attribution on HTTPS)

4.2 Signature Matching – `detector.rs`

`SignatureIndex` pre-builds three `HashSets` (IPs, domains, URLs) plus a nested `HashMap` for full Signature lookup. `match_flow()` checks src/dst IP, then hostname, then URL — all $O(1)$. Domains and URLs are normalized (lowercase, trailing-dot/slash stripped) for reliable matching.

4.3 Serialization – Cache & JSON Export

All core types (Flow, Signature, Detection, Verdict) derive `Serialize/Deserialize (serde)`. The threat-intel cache (`signatures_cache.json`) stores `{ fetched_at, signatures[] }` with a 1-hour TTL. `report::export_json` writes a pretty-printed `detections.json` containing every Detection for downstream analysis.

5. Features, Results & Testing

- Pcap ingestion via `pcap-file` + `ethers` (no root required).
- Offline mode (`SIG_IDS_OFFLINE=1`) loads `sample_signatures.json` for air-gapped demos.
- Unit tests cover matching (IP/domain/URL), aggregation (same/reverse direction), and classification thresholds.
- Sample run produces detections with Malicious verdicts for high-confidence AbuseIPDB IPs and Suspicious for URLhaus domains/URLs.

Example summary output:

```
[BLOCKED] 10.0.0.5 -> 1.2.3.4 matched Signature { ... confidence: 95 }  
=== Detection Summary === Total flows: 3 | Benign: 2 | Suspicious: 0 | Malicious: 1
```

6. Conclusion

Cognton-AI-SIG-IDS demonstrates a complete, production-oriented signature-based IDS pipeline in Rust. It combines safe low-level packet parsing, efficient $O(1)$ IOC matching, real-world threat-intelligence integration with caching, and clear reporting. The modular design (types → ingestion → aggregate → signatures → detector → classifier → report) makes the system easy to extend—e.g., adding live capture, more feeds, or payload-hash matching—while remaining reliable and performant under the constraints of the assignment environment.

Technology stack: Rust 1.75, pcap-file, ethers, ureq/native-tls, serde/serde_json, clap, anyhow, chrono, url.